

Experiment 10

Image Generation Using a Generative Adversarial Network (GAN)

Aim

Generate new handwritten-digit images using a generative adversarial network trained on MNIST.

Source and execution status

New implementation prepared from the supplied Experiment 10 topic. The supplied EXP10 folder contains no notebook. MNIST is the chosen demonstration dataset; training has not been run and no measured results are claimed.

Requirements

Python, PyTorch, torchvision, NumPy and Matplotlib. An internet connection is needed for the first MNIST download. A GPU is optional; CPU training takes longer.

Method

The generator maps 100 random values through dense layers of 128, 256 and 512 units to 784 tanh outputs. The discriminator maps flattened images through 512 and 256 units to one logit. Real images are normalized to $[-1, 1]$.

Training procedure

Alternate two updates for each batch. First train the discriminator on real images with label 1 and detached generated images with label 0. Then freeze discriminator parameters and train the generator to obtain label 1 for its generated images. Use BCEWithLogitsLoss and separate Adam optimizers; the discriminator has no final sigmoid.

Settings

Batch size: 128. Epochs: 20. Learning rate: 0.0002. Adam betas: (0.5, 0.999). Random seed: 42. Fixed noise samples allow visual comparison across epochs.

Run instructions

Execute the complete program below, or run the code cell in the accompanying local notebook. The main guard and `num_workers=0` support Windows and notebook execution. Outputs are written to `gan_outputs`.

Expected outputs - not measured results

The program saves an 8 x 8 sample grid after each epoch, a loss plot and a checkpoint. It displays the final generated images. Early samples may look noisy; inspect digit structure and diversity after training. Loss values alone do not establish image quality. If outputs repeat the same digit, investigate mode collapse.

Result status

Pending execution. Insert actual training logs, generated-image grids and observations after running the program. No sample images or numerical results have been fabricated.

Complete program

"""Experiment 10: Image generation using a GAN on MNIST.

New implementation authored from the supplied topic.

Training results have not been produced. Install torch, torchvision and matplotlib before running.

"""

```
from pathlib import Path
import random
import numpy as np
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms, utils
import matplotlib.pyplot as plt

def main():
    # 1. Reproducibility, device and experiment settings
    seed = 42
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
    device = torch.device(
        'cuda' if torch.cuda.is_available() else 'cpu'
    )
    latent_dim, batch_size, epochs = 100, 128, 20
    output = Path('gan_outputs')
    output.mkdir(exist_ok=True)
    print('Device:', device)

    # 2. Load MNIST and normalize pixels to [-1, 1]
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.5,), (0.5,))
    ])
    dataset = datasets.MNIST(
        './data', train=True, download=True,
        transform=transform
    )
```

```

# num_workers=0 is suitable for notebooks and
Windows.
loader = DataLoader(
    dataset, batch_size=batch_size, shuffle=True,
    num_workers=0, drop_last=True
)
print('Training images:', len(dataset))

# 3. Generator: random noise -> 28 x 28 grayscale
image
generator = nn.Sequential(
    nn.Linear(latent_dim, 128),
    nn.LeakyReLU(0.2),
    nn.Linear(128, 256),
    nn.BatchNorm1d(256),
    nn.LeakyReLU(0.2),
    nn.Linear(256, 512),
    nn.BatchNorm1d(512),
    nn.LeakyReLU(0.2),
    nn.Linear(512, 784),
    nn.Tanh()
).to(device)

# 4. Discriminator: image -> real/fake logit
discriminator = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 512),
    nn.LeakyReLU(0.2),
    nn.Dropout(0.3),
    nn.Linear(512, 256),
    nn.LeakyReLU(0.2),
    nn.Dropout(0.3),
    nn.Linear(256, 1)
).to(device)
# This loss applies sigmoid internally for
stability.
criterion = nn.BCEWithLogitsLoss()
opt_g = torch.optim.Adam(
    generator.parameters(), lr=0.0002,
    betas=(0.5, 0.999)
)
opt_d = torch.optim.Adam(
    discriminator.parameters(), lr=0.0002,
    betas=(0.5, 0.999)
)
fixed_noise = torch.randn(64, latent_dim,
device=device)

```

```

history_g, history_d = [], []

# 5. Alternate discriminator and generator updates
for epoch in range(epochs):
    generator.train()
    discriminator.train()
    total_g = total_d = 0.0
    for real_images, _ in loader:
        real_images = real_images.to(device)
        n = real_images.size(0)
        real_labels = torch.ones(n, 1,
device=device)
        fake_labels = torch.zeros(n, 1,
device=device)

        # D learns to distinguish real and
generated images.
        opt_d.zero_grad(set_to_none=True)
        noise = torch.randn(n, latent_dim,
device=device)
        fake_images = generator(noise).view(n, 1,
28, 28)
        loss_real = criterion(
            discriminator(real_images), real_labels
        )
        loss_fake = criterion(
            discriminator(fake_images.detach()),
fake_labels
        )
        loss_d = loss_real + loss_fake
        loss_d.backward()
        opt_d.step()

        # G learns to make D classify generated
images as real.
        # Freeze D's parameters while preserving
gradients to G.
        for parameter in
discriminator.parameters():
            parameter.requires_grad_(False)
        opt_g.zero_grad(set_to_none=True)
        loss_g = criterion(
            discriminator(fake_images), real_labels
        )
        loss_g.backward()
        opt_g.step()
        for parameter in

```

```

discriminator.parameters():
    parameter.requires_grad_(True)
    total_g += loss_g.item()
    total_d += loss_d.item()

    history_g.append(total_g / len(loader))
    history_d.append(total_d / len(loader))
    print(
        f'Epoch {epoch + 1}/{epochs}: '
        f'D loss={history_d[-1]:.4f}, '
        f'G loss={history_g[-1]:.4f}'
    )

    # 6. Save comparable samples with the same
    fixed noise.
    generator.eval()
    with torch.no_grad():
        samples = generator(fixed_noise).view(64,
1, 28, 28)
        samples = (samples + 1) / 2
        utils.save_image(
            samples, output / f'epoch_{epoch +
1:02}.png',
            nrow=8
        )

    # 7. Plot losses and show the final generated
    samples.
    plt.plot(range(1, epochs + 1), history_g,
label='Generator')
    plt.plot(range(1, epochs + 1), history_d,
label='Discriminator')
    plt.xlabel('Epoch')
    plt.ylabel('Mean training loss')
    plt.title('GAN training losses')
    plt.legend()
    plt.tight_layout()
    plt.savefig(output / 'training_losses.png',
dpi=160)
    plt.show()
    grid = utils.make_grid(samples.cpu(), nrow=8)
    plt.figure(figsize=(8, 8))
    plt.imshow(grid.permute(1, 2, 0).numpy())
    plt.axis('off')
    plt.title('Generated MNIST images')
    plt.tight_layout()
    plt.show()

```

```
torch.save(  
    {'generator': generator.state_dict(),  
     'discriminator': discriminator.state_dict(),  
     'latent_dim': latent_dim,  
     'generator_loss': history_g,  
     'discriminator_loss': history_d},  
    output / 'gan_checkpoint.pt'  
)  
print('Outputs saved in:', output.resolve())  
  
if __name__ == '__main__':  
    main()
```